

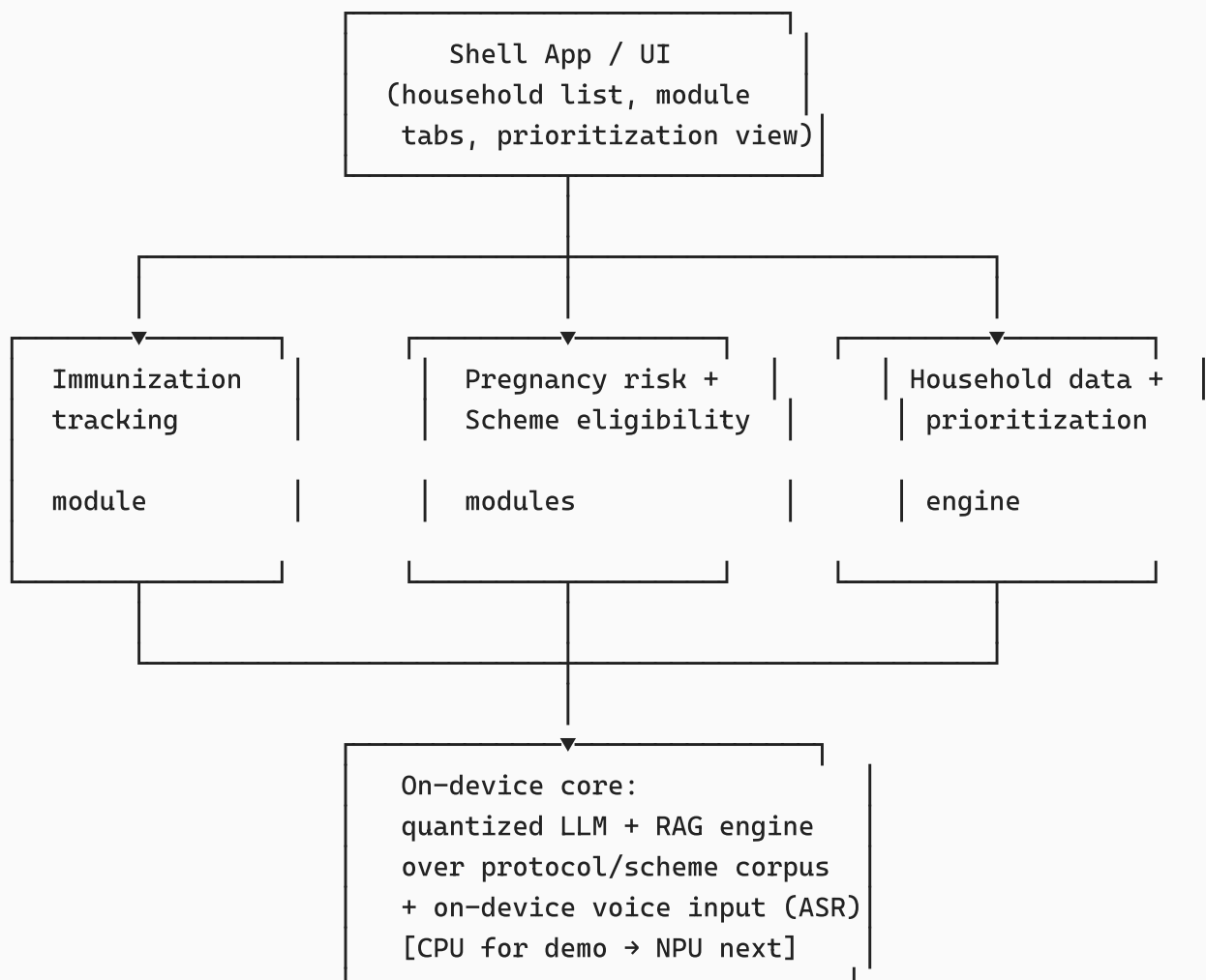
02_implementation_doc

Sahaya — Implementation Plan

3-person build, 4 days

Demo scope note: We do not have access to Snapdragon hardware for this build. Everything in this plan is built exactly as designed — same model, same runtime choice, same RAG pipeline, same app — except inference targets **CPU** for the demo instead of the Snapdragon NPU. All Day 1 "confirm NPU dispatch on real hardware" steps are replaced with "confirm CPU inference works end-to-end and is fast enough for a live demo." NPU dispatch verification moves to the next iteration once hardware is available. Everything else in this doc — schema, track split, module order, integration plan — remains unchanged.

1. Architecture overview



Every module calls the same on-device core with a different corpus slice and a different output schema. Building the core first and well is the single highest-leverage thing this team does — everything else is UI and data around it. The core is built with an NPU-capable runtime (see Section 2) but dispatched on CPU for this demo, so swapping the backend later is a configuration change, not a rewrite.

⚠️ **No hardcoding, no fallbacks — top priority across every track.** Nothing in this build should quietly substitute a real, live pipeline result with a hardcoded value, a scripted/pre-written answer, or a silent fallback triggered on failure — not in the core RAG pipeline, not in immunization/risk/eligibility logic, not in the multi-turn demo flow. If something is unreliable or fails during build/testing, treat that as a bug to fix or an open problem to raise with the team explicitly — not something to work around with a hardcoded stand-in. This applies for the whole 4 days, including Day 4 rehearsal.

★ **Showstopping feature — HIGH PRIORITY: multi-turn conversation.** The core pipeline must support holding a short conversation buffer (last 1–2 turns) so the ASHA worker can ask a follow-up question by voice — "what about the second dose then?" — and get a contextually correct, still-grounded answer, live, fully offline. This is the single feature most likely to make judges sit up: it's the difference between "a search bar with a mic" and "a real conversation with no signal and no server." Treat this as a first-class requirement of the core pipeline, not a stretch add — see Track A below and Section 4 of the Day 4 plan.

2. Day 1 — Shared foundation (all 3 people, together)

Don't split up yet. Day 1 output is the spine everything else plugs into.

- **Pick the on-device stack, run it on CPU for this build:**
 - Small quantized LLM candidate (e.g. a ~1–3B instruction-tuned model, INT4/INT8 quantized) — pick the same model you'd eventually want on the NPU, so nothing has to change later
 - Runtime: still choose one of ONNX Runtime, Google AI Edge / MediaPipe LLM Inference API, or llama.cpp — the same runtimes that support QNN/NPU delegates also run fine on CPU execution providers. Pick based on which one gives the easiest CPU path today and has a documented NPU delegate path for later, so the "next iteration" swap is a backend config change, not a re-architecture
 - Confirm CPU inference latency is acceptable for a live demo (this replaces "confirm NPU dispatch" as the Day 1 hardware validation step) — if it's too slow on CPU, that's your Day 1 signal to drop to a smaller/more aggressively quantized model
- **Define the household data schema** (SQLite, local only): household, mother/child records, dose history, ANC visit history, income/category fields needed for scheme matching
- **Stand up the shell app:** navigation shell with tabs/modules, wired to placeholder data for layout purposes only — treat this as strictly temporary scaffolding to unblock UI work,

and swap every screen to live pipeline/engine output before Day 4; nothing placeholder should still be on screen by demo day

- **Start the corpus:** pull real UIP immunization schedule, ANC/high-risk pregnancy protocol, and scheme eligibility rules (JSY, PMMVY, PMSMA, Ayushman Bharat, Anemia Mukd Bharat, RBSK) from official MoHFW/NHM sources — not secondary blogs — and structure them into retrievable chunks
- **Decide the RAG pipeline:** on-device embedding model + local vector store (even a simple in-memory/SQLite vector index is fine at this scale)

By end of Day 1: one person should be able to type a protocol question into a bare-bones interface and get a corpus-grounded answer, running fully offline on-device on CPU. That's the proof the core works before anyone builds a vertical on top of it. (NPU dispatch proof is deferred to the next iteration, once Snapdragon hardware is available — see the closing section of this doc.)

By end of Day 2: the core should accept a second, follow-up query that references the first exchange (e.g. "what about the second dose then?") and return a contextually correct grounded answer, generated genuinely through retrieval + inference against the live conversation buffer — this is the multi-turn showstopping feature and it needs early validation, not a Day 4 surprise. If context-holding retrieval turns out to be unreliable, treat that as a real engineering problem to solve (debug the query-rewrite/retrieval logic, try a different context-folding approach) — do not paper over it with a simplified or hardcoded substitute. If it genuinely can't be made reliable, that's a decision to surface to the team explicitly, not something to quietly work around.

3. Track split (Days 2–3)

Track A — Person 1: On-device core + Immunization module

- Owns the RAG engine, corpus indexing, and on-device inference pipeline end-to-end (CPU-backed for this build) — this person is on call for the other two tracks when their module needs a query against the core
- **Owns the multi-turn conversation feature (★ high priority, showstopping demo moment)** — extends the core's query interface to accept a short conversation buffer (last 1–2 turns) and thread it into retrieval, so a follow-up question resolves correctly against the prior exchange. Build and validate this by end of Day 2, ahead of Track C wiring it into the voice UI on Day 3–4
- Builds the immunization tracking module: dose-due-date logic against UIP schedule, overdue/upcoming flags, protocol Q&A surfaced through the core
- Simplest data shape of the three verticals — good anchor to get a full vertical working early and prove the pattern the other two will copy
- Keep the model/runtime choice and prompt/retrieval interface backend-agnostic (input/output contract unchanged whether the backend is CPU or NPU) so the next-iteration NPU swap doesn't touch anything above this layer

Track B — Person 2: Pregnancy risk + Scheme eligibility

- Pregnancy risk module: ANC checklist input → risk score/flag against maternal health protocol, with the *reason* for the flag pulled from the corpus (not a black-box score)
- Scheme eligibility module: rule-based matcher (income, category, delivery status, employment) against real scheme criteria, output explains *why* a household qualifies and what to do next
- These two share a lot of "match household facts against structured rules + explain via corpus" logic — building them together avoids duplicating that pattern

Track C — Person 3: Household prioritization + on-device voice input

- Prioritization engine: ranks the household list using signals from Tracks A and B (overdue doses, risk flags, missed ANC visits, staleness of last visit) — this is the integration point, so this person needs to sync with A and B daily on output formats
- On-device voice input (ASR): local speech-to-text for field queries, covering Hindi + at least 2 regional languages (pick based on team's testing access/demo location) — runs on CPU for this build, same as the core LLM — this is still a real differentiator (on-device, offline, multilingual) even before NPU dispatch is added
- **Wires the voice UI to the multi-turn conversation feature (★ high priority)** — once Track A's context-holding query interface is validated, this person builds the UI/UX for a live back-and-forth: mic stays reachable after the first answer, a lightweight visual cue shows the app is "still listening for a follow-up," and the conversation buffer resets cleanly per household. Sync with Track A daily on this — it's the top demo-flow risk alongside prioritization output
- Owns UX/intuitiveness once data is flowing: icon-first navigation, localized UI chrome (not just localized answers), urgency-coded household list, no login/setup friction, and a Day 3–4 usability check with someone outside the team

4. Day 4 — Integration, data, rehearsal

- Morning: merge all three tracks into the shell app, fix integration breakage
- Midday: build realistic synthetic demo **input** data — a believable village of ~15–20 households with varied, plausible-looking raw fields (dose dates, ANC readings, income/category, visit history). The overdue-dose flags, risk flags, scheme eligibility, and priority scores/reasons shown for these households must be *computed by actually running Track A/B/C's real modules on this input data* — never pre-written or manually assigned to look plausible. If the computed output for a synthetic household looks wrong or boring, fix the input data or the underlying logic, don't hand-edit the output
- Afternoon: rehearse the single narrative demo flow end-to-end in airplane mode, on the actual device, more than once — **rehearse the multi-turn conversation moment specifically and repeatedly**, using genuine, live follow-up questions each time (not a scripted or pre-agreed set). If general follow-up handling isn't fully reliable by Day 4,

that's a signal to go back and fix the underlying retrieval/context logic, not to script known-good exchanges as a stand-in — the demo should show the real pipeline working, not a rehearsed sequence dressed up as one

- **Buffer:** keep the last couple of hours unscheduled — something in the pipeline will need a fix at the last minute; budget for it rather than hoping you won't need it
- **Pitch note:** be ready to answer "is this running on the NPU?" honestly — the answer for this demo is "on-device, CPU-backed, fully offline; NPU dispatch is the next iteration once we have Snapdragon hardware." Have this framed as a roadmap point, not something to gloss over — judges tend to respect an honest, specific "what's next" more than an implied claim that doesn't hold up under a follow-up question.

5. Coordination notes

- Daily 15-minute sync (morning) to agree on data contracts between Track A/B's outputs and Track C's prioritization input — this is the seam most likely to break under time pressure
- Track A is the bottleneck dependency for A and B's protocol Q&A — get its interface (query in, grounded answer + source out) frozen by end of Day 1 so B can build against it without waiting
- Multi-turn conversation is the highest-priority showstopping feature and the interface between Track A (context-holding query logic) and Track C (voice UI) — get this contract frozen by end of Day 2, same urgency as the Day 1 core freeze, since it needs two full days of joint testing before demo day
- Keep the corpus small and curated rather than broad — a tight, accurate set of real protocols beats a large scraped one that might hallucinate on stage

6. Next iteration (post-demo, once Snapdragon hardware is available)

- Get hands-on time with the actual Snapdragon device/SDK; validate the chosen runtime's QNN (or equivalent) execution provider on it first, standalone, before touching app code
- Swap the core's inference backend from CPU to the NPU delegate — because the model/runtime choice in Section 2 was picked with this path in mind, this should be a backend/config change against the same query-in/answer-out interface, not a rewrite
- Verify actual NPU dispatch via profiling/logs (not just API selection) — treat silent CPU fallback as a build-blocking bug
- Re-run the Day-1-style latency/accuracy check to confirm the NPU path is actually faster/more efficient than the CPU path that shipped in this demo, and re-benchmark before making any performance claims publicly